Java Collections Framework

Further Programming

RMIT UNIVERSITY

Learning Outcomes



Describe the general concept of a collection in computer programming

Explain the difference between `List`, `Set` and `Map` and their implementations and when these are appropriate for a given programming problem. **Demonstrate** their use in Java code.

Demonstrate the use of Iterator and **for-each** loop in Java and the JCF

Describe the `java.lang.Object` equals() and `hashCode()` methods and **demonstrate** how these are used in JCF code

Discuss the Big-O notation and performances of different implementations

Describe Java Generics

Static versus dynamic types and abstraction (1)



When we instantiate an object in Java we actually specify two types: `List myList = new ArrayList();`

The type on the left of the '=' (instantiation operator) is called the static (*) or declared type (in the above example `List` i.e. the `java.util.List` interface)

This type is used by the compiler to type check any references where `myList` is used (e.g. you cannot assign a `List` to a `String`, you also cannot assign a `List` to an `ArrayList` since not all Lists are ArrayLists (although all ArrayList instances are Lists)

Only methods or attributes that exist in the static/declared type can be accessed

(regardless of the type on the right of '=')

Static versus dynamic types and abstraction (2)



Try to use the most Abstract type possible (highest in class hierarchy) to favor reuse and choose interfaces over classes where possible. **This applies to parameter types as well!**

The type on the right hand side is the dynamic or runtime type. It is the actual type of the object when instantiated during runtime and determines which actual implementation of methods are used (e.g. `ArrayList.add()` versus `LinkedList.add()`).

Having collections of interfaces or abstract types facilitates polymorphism by placing different dynamic/runtime types in a collection

(*) Not to be confused with the static java keyword!

Java Collections



Collection: An Object that groups multiple elements into a single unit

Typically represents data items belonging to a natural group, such as Library Catalogue (a collection of library materials and borrow-status)

To store, retrieve and manipulate data (e.g., through polymorphic operations), and to transmit data from one method to another

Java Collections Implementation in earlier versions (still supported)

1. `java.util.Vector` - collection of Java Objects (contents can be instantiated from different classes) without prior knowledge of the size
2. `java.util.Hashtable` – collection of name-value pairs, like a dictionary, easy lookup
3. `java.util.Stack` – LIFO structure, extension of Vector with push(), pop() etc.
4. `Array` – collection of objects of same type (class or primitive data) with known fixed size

Java Collections Framework



A “Unified Architecture” for representing and manipulating collections. Collections Framework contains:

Interfaces: abstract data types representing collections. Interfaces allow collections to be manipulated independently of the details of their representation. `java.util.Collection` etc.

Implementations: concrete implementations of the collection interfaces. In essence, these are reusable data structures. `java.util.ArrayList` etc.

Algorithms: methods that perform useful computations, like searching and sorting, on objects that implement collection interfaces. These algorithms are said to be polymorphic because the same method can be used on many different implementations of the appropriate collections interface. In essence, algorithms are reusable functionality. `java.util.Collections`, `java.util.Arrays` (note the “s” on the end!)

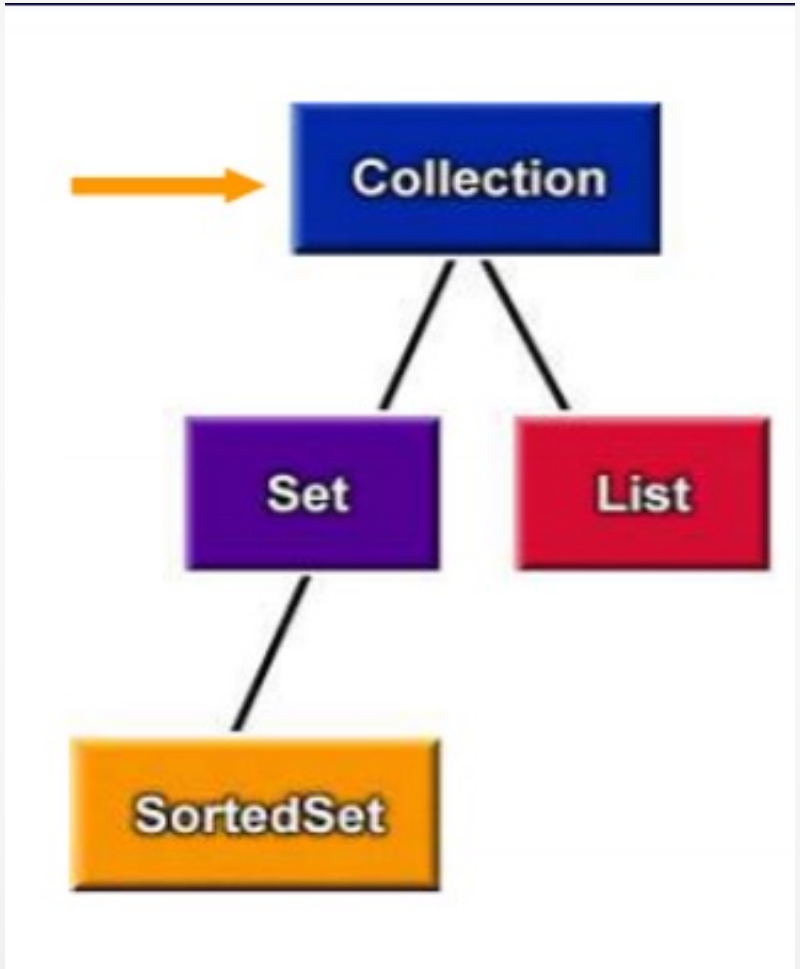
One of the basic features of OO Programming is **reusability**

Interface: Collection (1)



```
public interface Collection
{
        // Basic Operations
        int size();
        boolean isEmpty();
        boolean contains(Object element);
        boolean add(Object element); // Optional
        boolean remove(Object element); // Optional
        Iterator iterator();
       // Bulk Operations
        boolean containsAll(Collection c);
        boolean addAll(Collection c); // Optional
        boolean removeAll(Collection c); // Optional
        boolean retainAll(Collection c); // Optional
        void clear(); // Optional
        // Array Operations
        Object[] toArray();
        Object[] toArray(Object a[]);
}
```

Interface: Collection (2)



Iterator



Iterator interface allows looping through elements of a collection

Iterator also allows the caller to **remove** elements from the underlying collection during the iteration

```
public interface Iterator
{
 boolean hasNext();
 Object next();
 void remove(); // Optional
}
```

Enumeration (Old)



Enumeration Interface is replaced by Iterator interface

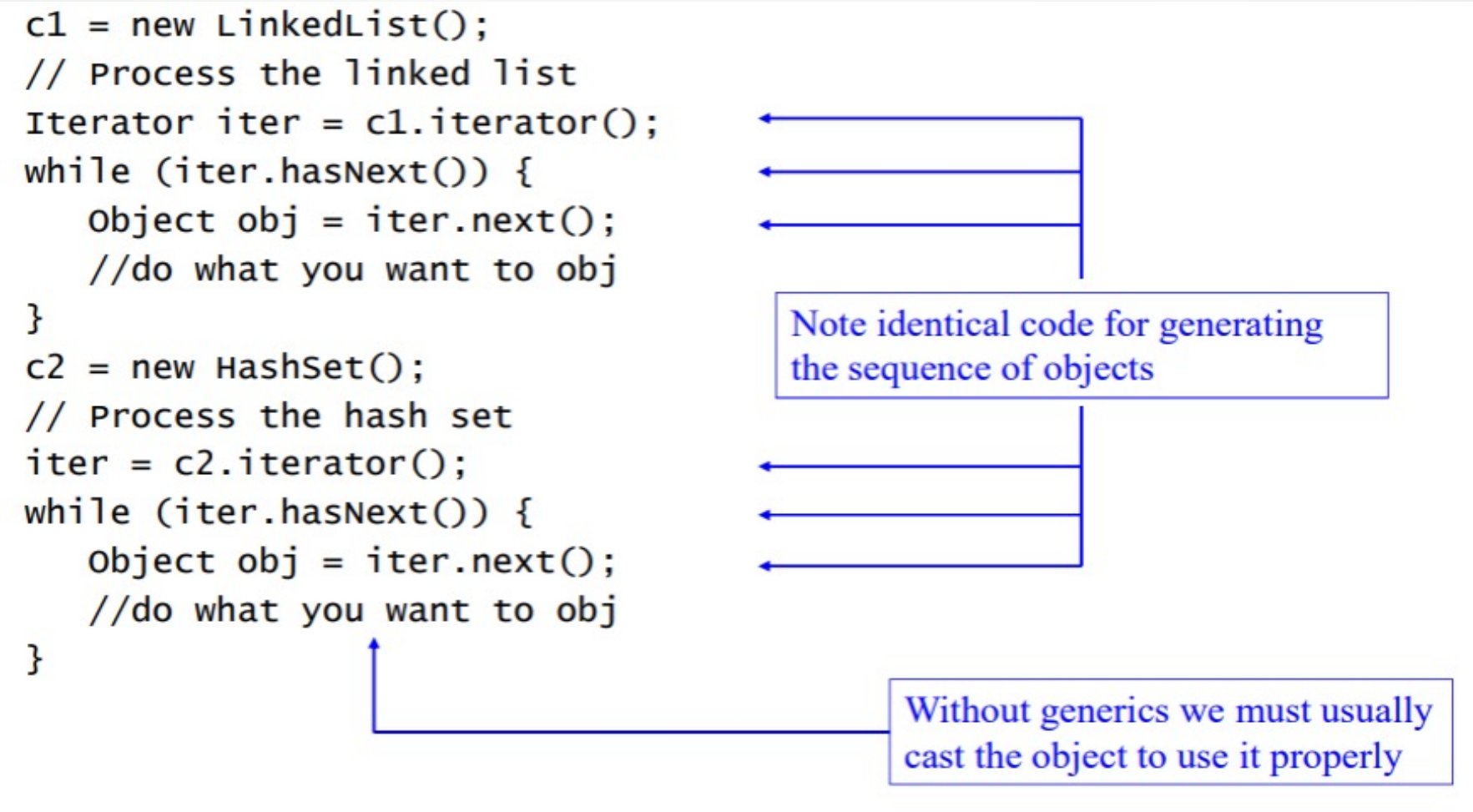
An object that implements the Enumeration interface generates a series of elements, one at a time. Successive calls to the nextElement method return successive elements of the series. e.g., StringTokenizer

Two methods:

```
boolean hasMoreElements()

Object nextElement()
```

Iterator Example



Use a for-each Loop instead of an Iterator



```
c1 = new LinkedList();
// Process the linked list
Iterator iter = c1.iterator();
while (iter.hasNext()) {
   Object obj1 = iter.next();
   //do what you want to obj1
}
c2 = new HashSet();
// Process the hash set
iter = c2.iterator();
while (iter.hasNext()) {
   Object obj2 = iter.next();
   //do what you want to obj2
}
```

```
c1 = new LinkedList();
// Process the linked list
for(Object obj1: c1)
   //do what you want to obj1
}


c2 = new HashSet();
// Process the hash set
for(Object obj2: c2) {
   //do what you want to obj2
}

Classes that implement
java.lang.Iterable interface and
arrays can be used in foreach as
well as Arrays
```

Interface: List (1)



A **List** is an ordered **Collection** (sometimes called a sequence). Lists may contain duplicate elements

In addition to the operations inherited from Collection, the List interface includes operations for:

1. **Positional Access (indexed):** manipulate elements based on their numerical position in the list.
2. **Search**: search for a specified object in the list and return its numerical position.
3. **List Iteration**: extend Iterator semantics to take advantage of the list's sequential nature.
4. **Range-view:** perform arbitrary range operations on the list

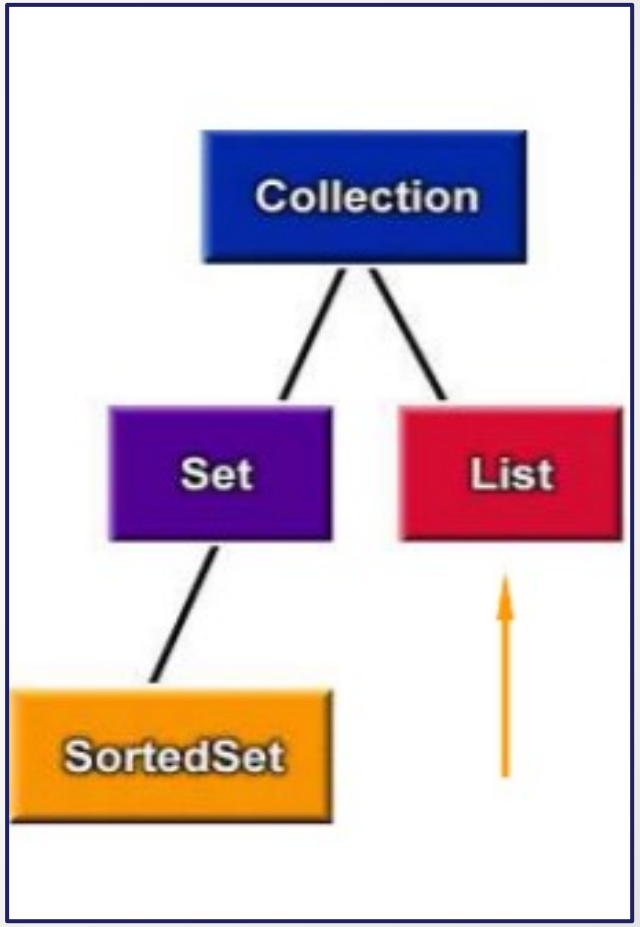
Interface: List (2)

JDK Standard Implementation (examples)

ArrayList – best general purpose and stored in a resizable array

LinkedList - better performance under certain circumstances and implements Deque interface

Vector – retrofitted to implement List, is also Thread safe. If a thread-safe implementation is not needed, it is recommended to use ArrayList in place of Vector



Interface: List (3)



```
public interface List extends Collection {
   // Positional Access
   Object get(int index);
   Object set(int index, Object element); // Optional
   void add(int index, Object element);   // Optional
   Object remove(int index);              // Optional
   abstract boolean addAll(int index,
                           Collection c); // Optional

   // Search
   int indexOf(Object o);
   int lastIndexOf(Object o);

   // Iteration
   ListIterator listIterator();
   ListIterator listIterator(int index);

   // Range-view
   List subList(int from, int to);
}
```

```
public interface ListIterator
   extends Iterator {

   boolean hasNext();
   Object next();

   boolean hasPrevious();
   Object previous();

   int nextIndex();
   int previousIndex();

   void remove();          // Optional
   void set(Object o);     // Optional
   void add(Object o);     // Optional
}
```

Most of the polymorphic algorithms in the **JCF** classes (e.g. java.util.Collections) apply specifically to List not Collection.

- `sort(List)`
- `shuffle(List)`
- `reverse(List)`
- `fill(List, Object)`
- `copy(List dest, List src)`
- `binarySearch(List, Object)`

ArrayList Class



- Grows dynamically
- Implements `List` methods
- Random access to data
- Faster at accessing elements anywhere in the list
- Slow at inserting (deleting) elements in the middle due to copying overhead

LinkedList Class



Grows dynamically

Implements `List` methods

Sequential access to data

Faster at inserting (deleting) elements anywhere in the list

Slow at accessing elements because the list has to be traversed sequentially

Interface: Set (1)



A **Set** is a **Collection** that cannot contain duplicate elements
Set models the mathematical set abstraction
The Set interface extends Collection and contains no methods other than those inherited from Collection
Set also adds a stronger contract on the behavior of the equals and hashCode operations (inherited from Object), allowing Set objects with different implementation types to be compared meaningfully. Two Set objects are equal if they contain the same elements

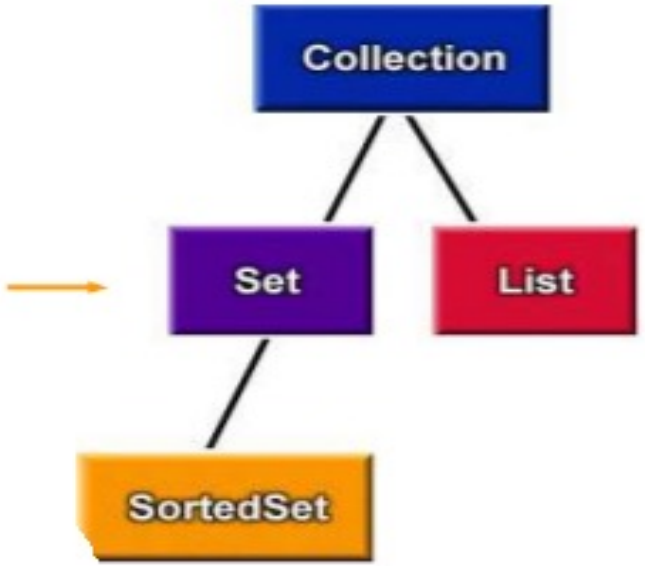
JDK Standard Implementations

HashSet – best performing and stored in hash table but random ordering

TreeSet - guarantees ordering and stored in red-black tree

LinkedHashSet – ordered by insertion order

Interface: Set (2)



What are the common Set Operations?

Subset	`s1.containsAll(s2)`
Union	`s1.addAll(s2)`
Intersection	`s1.retainAll(s2)`
Set Difference	`s1.removeAll(s2)`

Interface: Set (3)



`Set` extends `Collection` but has no new methods

- But classes implementing `Set` interface must ensure no duplicates, i.e., they cannot contain e1 and e2 if e1.equals(e2) is true

`HashSet` provides a concrete implementation of `Set`. Elements in `HashSet` are not ordered

`LinkedHashSet` extends `HashSet` with a linked list and supports an ordering. Elements can be retrieved in order of insertion

`TreeSet` class provide a concrete implementation of `SortedSet`. It guarantees elements are sorted. Objects inserted must be comparable with each other

Set vs. Collection add() Method



`java.util.Collection.add(Object e)`

Ensures that this collection contains the specified element (optional operation). Returns true if this collection changed as a result of the call. (Returns false if this collection does not permit duplicates and already contains the specified element.)

`java.util.Set.add(Object e)`

Adds the specified element to this set if it is not already present (optional operation). More formally, adds the specified element e to this set if the set contains no element `e2` such that e.equals(e2). If this set already contains the element, the call leaves the set unchanged and returns false

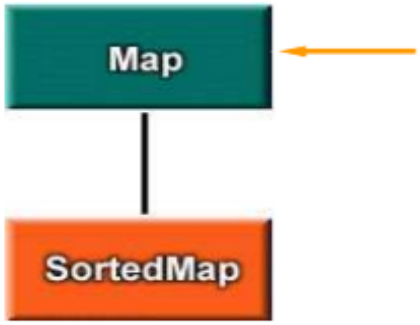
Interface: Map (1)



A **Map** is a collection that maps keys to values
A map cannot contain duplicate keys: Each key can map to at most one value
JDK Standard Implementation (examples)

- HashMap – best performing and stored in hash table
- TreeMap – guarantees ordering and stored in red-black tree
- LinkedHashMap – ordered by insertion order
- Hashtable – retrofitted to implement Map

Interface: Map (2)




```
public interface Map
{
    // Basic Operations
    Object put(Object key, Object value);
    Object get(Object key);
    Object remove(Object key);
    boolean containsKey(Object key);
    boolean containsValue(Object value);
    int size();
    boolean isEmpty();

    // Bulk Operations
    void putAll(Map t);
    void clear();

    // Collection Views
    public Set keySet();
    public Collection values();
    public Set entrySet();

    // Interface for entrySet elements
    public interface Entry
    {
        Object getKey();
        Object getValue();
        Object setValue(Object value);
    }
}
```

Map Implementations



HashMap

- Efficient for locating a value, inserting/deleting mapping
- Entries not ordered
- Pre JDK 1.2 equivalent is HashTable (redesigned to fit JCF but methods retained for compatibility)

LinkedHashMap

- Extends HashMap with a linked list
- Can be accessed in order of insertion/access

TreeMap

- Implements the interface SortedMap and efficient for accessing maps in sorted order
- Comparison can be done using the compareTo() method (default) or by passing a Comparator to the TreeMap constructor

Queue and Deque



`java.util.Queue` - typically a FIFO (first-in, first-out) structure but may be ordered arbitrarily, for example by priority

- Removal is always from the head of the queue via a call to `remove() or poll()`
- In a FIFO queue elements are inserted at the tail of the queue but other kinds of queues may use different placement rules
- Implementations: see for example `LinkedList, PriorityQueue`

`java.util.Deque` - can be used as both FIFO (first-in, first-out) and LIFO (last-in, first-out)

- Extends the `Queue` interface
- New elements can be inserted, retrieved and removed at both ends
- Implementations: see for example `LinkedList`, `ArrayDeque`, `LinkedBlockingDeque`

equals() and hashCode()



`java.util.Collection` operations use the `java.lang.Object.equals(...)` and `hashCode()` methods

If object1 and object2 are equal according to their equals() method, they must also have the same hash code

If object1 and object2 have the same hash code they do NOT have to be equal (but may be)

The default Object implementations give you a very strict interpretation: only equal if comparing the same object

Most Java API classes (e.g., String) already provide suitable implementations

As a rule if you override one you should override both however especially if you are using a class as a key you must override these methods together correctly **(or more simply use both Object defaults for strict comparison)**

Comparable and Comparator Interface (1)



If a sorted Collection consists of String elements, it will be sorted into lexicographic (alphabetical) order. If it consists of Date elements, it will be sorted into chronological order. How?

Both String and Date implements the **Comparable** interface that provides a **natural ordering** for a class, which allows objects of that class to be sorted automatically

For a list whose elements do not implement Comparable, `Collections.sort(list)` will throw a **`ClassCastException`**

For a list whose elements cannot be compared to one another, `Collections.sort(list, comparator)` will throw a **`ClassCastException`**

Comparable and Comparator Interface (2)



Elements that can be compared to one another are called **mutually comparable**. While it is possible to have elements of different types be mutually comparable, none of the JDK types (Byte, Character, Long, Integer, Short, Double, Float, String, Date) permit inter-class comparison

```
public interface Comparable
{
      public int compareTo(Object o);
}

public interface Comparator
{
      int compare(Object o1, Object o2);
}
```

Interface: SortedSet (1)

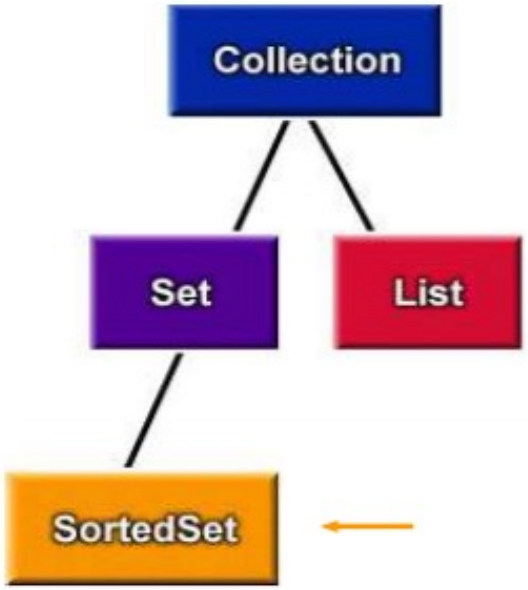


A **SortedSet** is a **Set** that maintains its elements in ascending order, sorted according to the elements' **natural order**, or according to a Comparator provided at SortedSet creation time

In addition to the normal Set operations, the SortedSet interface provides operations for:

- **Range-view:** Performs arbitrary range operations on the sorted set
- **Endpoints:** Returns the first or last element in the sorted set
- **Comparator access**: Returns the Comparator used to sort the set

Interface: SortedSet (2)




```
public interface SortedSet extends Set {
    // Range-view
    SortedSet subSet(Object fromElement, Object toElement);
    SortedSet headSet(Object toElement);
    SortedSet tailSet(Object fromElement);

    // Endpoints
    Object first();
    Object last();

    // Comparator access
    Comparator comparator();
}
```

Interface: SortedMap (1)

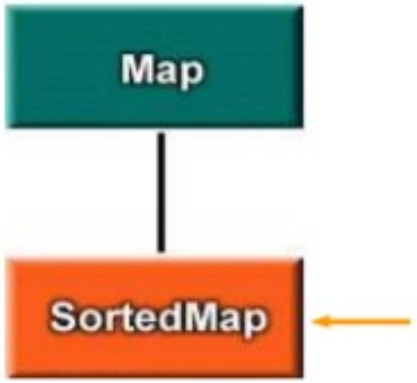


SortedMap is a **Map** that maintains its elements in ascending order, sorted according to the keys **natural order**, or according to a Comparator provided at SortedMap creation time

In addition to the normal Map operations, the SortedMap interface provides operations for:

- **Range-view**: Performs arbitrary range operations on the sorted map
- **Endpoints**: Returns the first or last key in the sorted map
- **Comparator access**: Returns the Comparator used to sort the map

Interface: SortedMap (2)




```
public interface SortedMap extends Map {
    // Range-view
    SortedMap subMap(Object fromKey, Object toKey);
    SortedMap headMap(Object toKey);
    SortedMap tailMap(Object fromKey);

    // Endpoints
    Object first();
    Object last();

   // Comparator access
    Comparator comparator();

}
```

General Purpose Implementations



		Implementations			
		Hash Table	Resizable Array	Balanced Tree	Linked List
Interfaces	Set	HashSet		TreeSet	
	List		ArrayList, Vector		LinkedList
	Map	HashMap		TreeMap	

(Reference : Java Tutorial – Joshua Bloch, collection trail)

The Big-O Notation (1)



A common way of assessing algorithm complexity (performance) is **Big-O** notation. This is an evaluation of how processing time or memory usage increases as the volume of data (N) increases

Linear search goes through every item in the collection. Therefore, running time has a direct linear relationship to volume of data. We call this a performance of **O(N)**, where N is the volume of data. This means that the complexity of the search is linearly related to the size of the data. Examples of Big-O(N): N, 2N, 1/2 N, N + 3, but not N^2 , N^3 , Log(N), 10^N

The Big-O Notation (2)



Big-O means that the complexity is an approximation of the specified N a Linear Search should stop when the correct key is found.

- **Best** case is when the key is in the first position. A performance where only one operation is needed is O(1)
- **Worst** case is when the key is in the last position and all items must be examined**: O(N).**
- **Average** case is when the key is about halfway, giving a performance of **O(N/2)** (asymptotically, O(N/2) is the same as O(N))

Performance Summary (1)



List implementations:

	get	add	contains	next	remove(0)	iterator.remove
ArrayList	O(1)	O(1)	O(n)	O(1)	O(n)	O(n)
LinkedList	O(n)	O(1)	O(n)	O(1)	O(1)	O(1)
CopyOnWrite-ArrayList	O(1)	O(n)	O(n)	O(1)	O(n)	O(n)

Set implementations:

	add	contains	next	notes
HashSet	O(1)	O(1)	O(h/n)	h is the table capacity
LinkedHashSet	O(1)	O(1)	O(1)	
CopyOnWriteArraySet	O(n)	O(n)	O(1)	
EnumSet	O(1)	O(1)	O(1)	
TreeSet	O(log n)	O(log n)	O(log n)	
ConcurrentSkipListSet	O(log n)	O(log n)	O(1)	

- Reference: https://stackoverflow.com/questions/559839/big-o-summary-for-java-collectionsframework-implementations

Performance Summary (2)



Map implementations:

	get	containsKey	next	Notes
HashMap	O(1)	O(1)	O(h/n)	h is the table capacity
LinkedHashMap	O(1)	O(1)	O(1)	
IdentityHashMap	O(1)	O(1)	O(h/n)	h is the table capacity
EnumMap	O(1)	O(1)	O(1)	
TreeMap	O(log n)	O(log n)	O(log n)	
ConcurrentHashMap	O(1)	O(1)	O(h/n)	h is the table capacity
ConcurrentSkipListMap	O(log n)	O(log n)	O(1)	

Queue implementations:

	offer	peek	poll	size
PriorityQueue	O(log n)	O(1)	O(log n)	O(1)
ConcurrentLinkedQueue	O(1)	O(1)	O(1)	O(n)
ArrayBlockingQueue	O(1)	O(1)	O(1)	O(1)
LinkedBlockingQueue	O(1)	O(1)	O(1)	O(1)
PriorityBlockingQueue	O(log n)	O(1)	O(log n)	O(1)
DelayQueue	O(log n)	O(1)	O(log n)	O(1)
LinkedList	O(1)	O(1)	O(1)	O(1)
ArrayDeque	O(1)	O(1)	O(1)	O(1)
LinkedBlockingDeque	O(1)	O(1)	O(1)	O(1)

Advantages of JCF



- Reduces programming effort
- Increases speed and quality
- Allows interoperability among unrelated code
- Reduces learning effort
- Reduces design effort
- Fosters software reuse

Java Generics



Added since **java 5** with the purpose of adding an extra layer of abstraction over types and reducing incompatibility bugs

Generics means **parameterized types.** Allow class types to be a parameter to methods, classes, and interfaces to create classes that work with multiple data types

Generic Methods, Generic Classes, Generic Interface

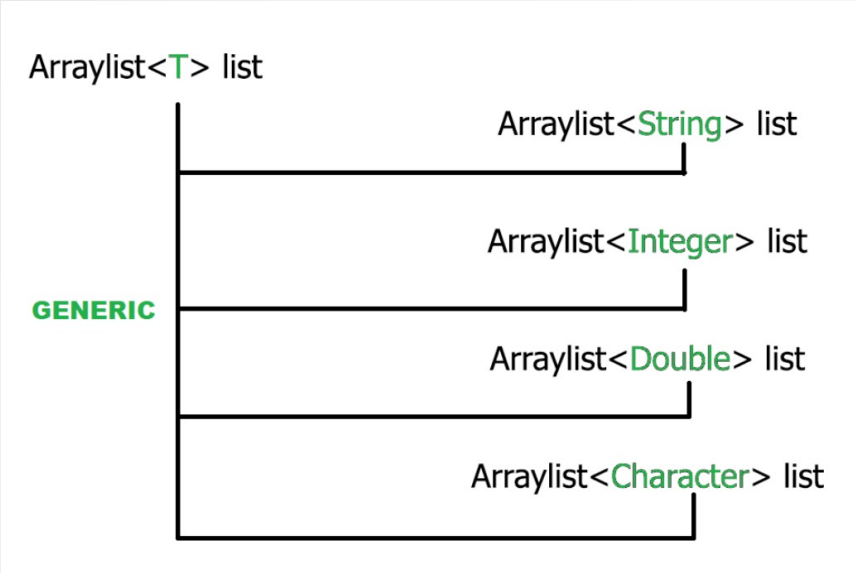
Use <> to specify parameter types in generic class creation

Only work with reference type, not primitive data types

Can bound type parameters

Provide Type-Safety and resolve Type-Casting problems

Generic Classes



Common parameters:

1. **T - Type** (any Wrapper class: String, Integer, Long, Float, …)
2. **E – Element** (element – commonly used in Collection Framework)
3. **K – Key**
4. **V – Value**
5. **N – Number** (Integer, Double, Float, …)
6. **U,S,I,G, …**

```
class Dictionary<K, V> {
    private K key;
    private V value;

    public Dictionary(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() {
        return key;
    }

    public void setKey(K key) {
        this.key = key;
    }

    public V getValue() {
        return value;
    }

    public void setValue(V value) {
        this.value = value;
    }
}
```

Generic Method



```
public <T> List<T> fromArrayToList(T[] a) {
    return Arrays.stream(a).collect(Collectors.toList());
}
```

```
public static <T, G> List<G> fromArrayToList(T[] a, Function<T, G> mapperFunction) {
    return Arrays.stream(a)
      .map(mapperFunction)
      .collect(Collectors.toList());
}
```

The <*T*> in the method signature implies that the method will be dealing with generic type *T*. This is needed even if the method is returning void.

Generic Interface



```
interface writer<T> {
    void update(T obj);
    void delete(T obj);
    void write(T obj);

}

class Book<T> implements writer<T> {

    @Override
    public void update(T obj) {
        // do something
    }

    @Override
    public void delete(T obj) {
        // do something
    }

    @Override
    public void write(T obj) {
        // do something
    }

}
```

```
public class Demo {
    public static void main(String[] args) {
        Book<String> t = new Book<String>();
        t.write("Add data to book");
    }
}
```

Bounded Generics and Wildcards



Type parameters can be bounded. For example, we can specify that a method accepts a type and all its subclasses (upper bound) or a type and all its superclasses (lower bound)

Wildcards (?) are referred as an unknown type

Upper bound:

```
public <T extends Number> List<T> fromArrayToList(T[] a) {
    ...
}
```

Lower bound:

```
List<? super Car> cars = new ArrayList <Vehicle>();
```

A type can also have multiple upper bounds

```
<T extends Number & Comparable>
```

Thank you

COSC2440 teaching team

RMIT UNIVERSITY